

数据库系统原理期末大作业

实验 7：NBA 数据分析智能体的设计与实现

项目	内容
课程	数据库系统原理
项目主题	基于 NBA 公开数据集的数据分析智能体
技术路线	Python + pandas + SQLite + LangChain + DeepSeek API + Streamlit
提交形式	RAR 压缩包，包含报告、代码、数据或数据获取说明、运行截图

角色	姓名	学号	任务分工
组员	邹传涛	37220232203970	环境搭建、报告整合、提交材料检查
组长	何后天	37220232203673	项目统筹、数据获取、pandas 清洗、SQLite 建库与验证查询
组员	彭泽喜	37220232203792	LangChain Data Agent、DeepSeek API 接入、Streamlit 可视化

说明：DeepSeek API Key 仅保存在本地 .env 文件中，报告和提交材料不包含真实密钥。

一、实验目的与总体要求

本实验围绕公开 NBA 篮球比赛数据集，设计并实现一个能够回答业务分析问题的数据智能体。项目覆盖真实数据集下载、抽样与清洗、关系数据库建模、SQL 查询、可视化展示和大模型辅助分析的完整流程。

- 使用 Python 和 pandas 完成数据读取、清洗、字段转换和统计概要生成。
- 使用 SQLite 建立关系数据库，提供建表、导入、索引和验证查询。
- 使用 LangChain 连接数据库，并结合 DeepSeek API 完成自然语言问题到 SQL 查询再到分析解释的流程。
- 使用 Streamlit 展示自然语言提问、SQL、查询结果、自然语言解释和图表。
- 报告中说明大模型使用场景、关键提示词、人工修改内容和系统复现步骤。

二、开发环境与工具

类别	说明
操作系统	macOS/Windows
Python	3.10.14
数据处理	pandas 2.3.3
数据库	SQLite，本地文件 db/nba_analysis.sqlite
Agent 框架	LangChain 1.3.11, langchain-openai 1.3.3
大模型 API	DeepSeek API，OpenAI 兼容 base_url=https://api.deepseek.com
界面与可视化	Streamlit 1.58.0, Plotly 6.8.0, Matplotlib 3.10.9
依赖管理	.venv 虚拟环境和 requirements.txt
版本控制	Git，本项目已分阶段提交 4 次 commit

三、数据集来源与字段说明

本项目选用 Kaggle 公开数据集 nathanlauga/nba-games。该数据集包含 2003 到 2022 年左右的 NBA 比赛、球队、球员、排名和球员单场技术统计数据，适合数据库课程中的多表建模、关联查询和业务分析。

原始文件	用途	主要字段
games.csv	比赛事实表	比赛日期、赛季、主客队、比分、命中率、篮板、助攻、主队是否获胜等。
games_details.csv	球员单场统计	球员、球队、上场时间、投篮、三分、罚球、篮板、助攻、抢断、盖帽、得分等。
players.csv	球员信息	球员姓名、球员 ID、所属球队、赛季。
ranking.csv	球队排名	球队、日期、赛季、胜负场、胜率、主客场战绩等。
teams.csv	球队信息	球队 ID、缩写、城市、昵称、主场球馆、建队年份等。

四、数据预处理

数据预理由 `src/preprocess.py` 完成。脚本读取 `data/raw` 下的 5 个 CSV 文件，统一字段名、转换类型、处理缺失值和重复记录，并输出清洗后的 CSV 到 `data/processed`。

- 字段名统一转为小写，避免 SQL 查询时大小写混乱。
- 日期字段转为 `YYYY-MM-DD` 文本格式，便于 SQLite 存储和排序。
- 比分、命中率、篮板、助攻、抢断、盖帽、得分等字段统一转为数值类型。
- 球员未上场记录不删除，而是保留并用 `played` 字段标记，避免破坏比赛名单完整性。
- 计数类技术统计缺失值按 0 处理；命中率类字段无法计算时保留为空。
- 按照业务主键去除明显重复记录，例如 `game_id` 或 `game_id + player_id`。

清洗前后统计如下。

表	原始行数	清洗后行数	删除重复行数	清洗前缺失单元格	清洗后缺失单元格
teams	30	30	0	4	4
games	26651	26622	29	1188	1287
players	7228	7228	0	0	0
rankings	210342	210313	29	206352	206323
game_details	668628	668339	289	3804854	2268343

五、数据库设计与建库

数据库使用 SQLite，建库脚本为 `src/build_database.py`。该脚本会重新生成 `db/nba_analysis.sqlite`，创建表结构、导入清洗后的 CSV、建立索引，并输出验证查询结果。

表名	主键/唯一约束	说明
teams	team_id	球队维表，保存球队城市、昵称、缩写、主场、建队年份等。
games	game_id	比赛事实表，保存赛季、日期、主客队、比分、命中率和主队胜负。
players	player_id + team_id + season	球员赛季信息表，保存球员与球队、赛季关系。
rankings	自增 id	球队排名表，保存球队某日期胜负场、胜率、主客场战绩。
player_game_stats	自增 id，唯一索引 game_id + player_id	球员单场技术统计表，关联比赛、球队和球员。

常用查询字段建立了索引，包括 `games(season)`、`games(game_date)`、`games(home_team_id)`、`games(visitor_team_id)`、`player_game_stats(game_id)`、`player_game_stats(player_id)`、`player_game_stats(team_id)`、`rankings(team_id)`、`rankings(standings_date)`。

数据库导入后行数验证如下。

table_name	row_count
teams	30
games	26622
players	7228
rankings	210313
player_game_stats	668339

部分建表示例和索引如下。

```
CREATE TABLE games (  
    game_date TEXT,  
    game_id INTEGER PRIMARY KEY,  
    home_team_id INTEGER,  
    visitor_team_id INTEGER,  
    season INTEGER,  
    pts_home REAL,  
    pts_away REAL,  
    fg_pct_home REAL,  
    fg_pct_away REAL,  
    home_team_wins INTEGER,  
    total_points REAL  
);  
  
CREATE INDEX idx_games_season ON games(season);  
  
CREATE INDEX idx_player_stats_player ON player_game_stats(player_id);
```

六、Data Agent 设计与实现

数据智能体由 src/agent.py 实现。系统采用 LangChain 的 SQLiteDatabase 读取 SQLite schema，使用 langchain-openai 的 ChatOpenAI 以 OpenAI 兼容方式接入 DeepSeek API。

1. 用户在 Streamlit 页面输入中文业务问题。
2. 系统读取 SQLite 数据库表结构，并把 schema 和用户问题一起传给 DeepSeek。
3. DeepSeek 按提示词生成一条 SQLite 只读 SQL。
4. src/sql_safety.py 校验 SQL，只允许 SELECT 或 WITH，禁止 INSERT、UPDATE、DELETE、DROP、ALTER、CREATE、PRAGMA 等危险语句。
5. SQL 校验通过后连接 SQLite 执行查询，返回真实结果表。
6. 系统将自然语言问题、SQL 和结果样例传给 DeepSeek，生成中文业务分析结论。
7. Streamlit 页面展示问题、SQL、结果表格、分析结论和图表。

项目	内容
SQL 生成提示词	要求模型只输出 SQLite SQL，不输出 Markdown 或解释；只能使用 SELECT/WITH；字段必须来自 schema。
结果解释提示词	要求模型只基于用户问题、SQL 和查询结果前 20 行作答，禁止编造查询结果之外的事实。

人工修改内容	人工补充了 SQL 安全检查、默认 LIMIT、错误提示和 Streamlit 展示逻辑，避免模型生成危险 SQL 或凭常识回答。
--------	---

核心调用流程示意如下。

```
llm = ChatOpenAI(
    model=os.getenv("DEEPSEEK_MODEL", "deepseek-v4-flash"),
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com"),
    temperature=0,
)
sql = generate_sql(question, schema_text, llm)
safe_sql = ensure_limit(sql, default_limit=100)
data = pd.read_sql_query(safe_sql, sqlite_connection)
analysis = explain_result(question, safe_sql, data, llm)
```

七、Streamlit 界面与可视化

界面由 src/app.py 实现，页面标题为 NBA 数据分析智能体。左侧显示数据库连接状态和示例问题，主区域显示用户输入框、生成 SQL、SQL 查询结果、智能体解释和固定业务分析图表。

界面如下：



- 数据库不存在时，页面提示先运行 preprocess.py 和 build_database.py。
- 缺少 DEEPSEEK_API_KEY 时，智能体部分给出明确错误，不会崩溃。
- 固定图表不依赖大模型，能够直接用 SQLite 真实查询结果展示。

- 内置 5 个业务问题，便于课堂验收和报告截图复现。

固定业务分析图表：



图 1 赛季场均总得分趋势。2003 赛季场均总得分为 186.00，后续赛季整体呈上升趋势。



图 2 球队场均得分 Top 10。Warriors、Suns、Nuggets 位居前列。



图 3 主队胜率随赛季变化。多数赛季主队胜率约在 57% 到 61% 之间。

八、业务分析案例

案例 1：每个赛季比赛数量

自然语言问题：每个赛季的比赛数量是多少？

输入中文业务问题

每个赛季的比赛数量是多少？

运行分析

智能体生成或等价执行 SQL：

生成的 SQL

```
SELECT season, COUNT(*) FROM games GROUP BY season
LIMIT 100
```

SQL 查询结果节选：

SQL 查询结果

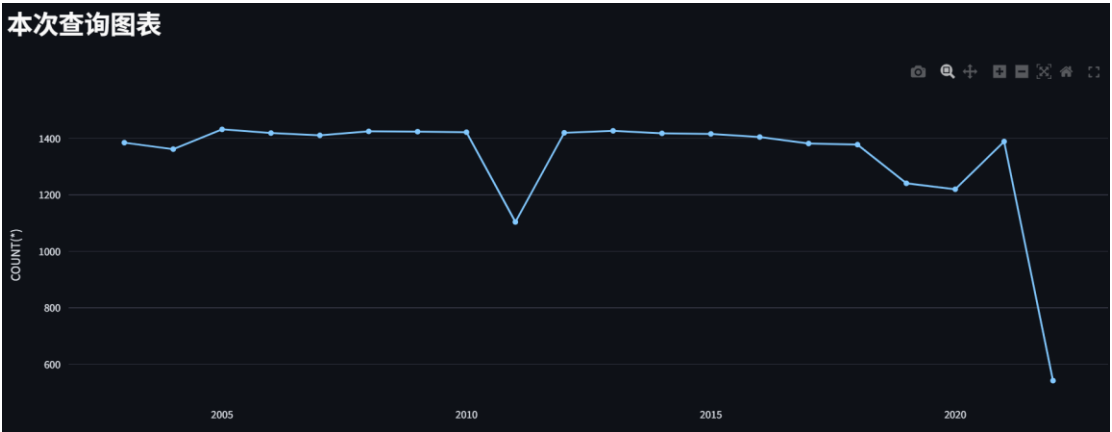
	season	⋮	COUNT(*)
0	2003		1385
1	2004		1362
2	2005		1432
3	2006		1419
4	2007		1411
5	2008		1425
6	2009		1424
7	2010		1422
8	2011		1104
9	2012		1420

自然语言解释：

智能体分析结论

根据查询结果，2003至2022年间各赛季比赛数量存在波动，多数赛季维持在1380至1430场之间。其中2011年（1104场）和2020年（1220场）明显偏低，可能与停摆或疫情有关；2022年仅542场，可能因数据未完整收录。整体来看，完整赛季的比赛数量通常在1400场左右。

本次查询图表：



案例 2：每个赛季主队胜率

自然语言问题：每个赛季主队胜率是多少？

输入中文业务问题

每个赛季主队胜率是多少？

运行分析

智能体生成或等价执行 SQL：

生成的 SQL

```
SELECT season, AVG(home_team_wins * 1.0) AS home_win_pct FROM games GROUP BY season ORDER BY season LIMIT 100
```

SQL 查询结果节选：

SQL 查询结果

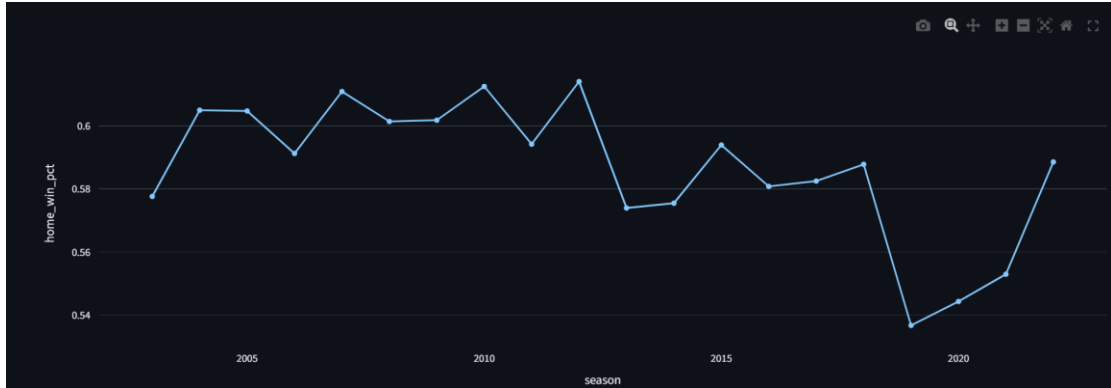
	↑ season	home_win_pct
0	2003	0.5776
1	2004	0.605
2	2005	0.6047
3	2006	0.5913
4	2007	0.6109
5	2008	0.6014
6	2009	0.6018
7	2010	0.6125
8	2011	0.5942
9	2012	0.6141

自然语言解释：

智能体分析结论

根据2003至2022赛季的数据，所有赛季的主队胜率均超过50%，说明主场优势普遍存在。其中2011-12赛季胜率最高（约61.4%），而2019-20赛季最低（约53.7%），可能与赛程变化（如疫情导致的空场）有关。整体来看，主队胜率在57%至61%之间波动，无长期单调趋势。

本次查询图表：



案例 3：球队场均得分排名

自然语言问题：哪些球队场均得分最高？

输入中文业务问题

哪些球队场均得分最高？

运行分析

智能体生成或等价执行 SQL：

生成的 SQL

```
SELECT t.nickname, t.team_id, AVG(pts) as avg_pts
FROM (
  SELECT team_id_home AS team_id, pts_home AS pts FROM games WHERE game_status_text = 'Final'
  UNION ALL
  SELECT team_id_away AS team_id, pts_away AS pts FROM games WHERE game_status_text = 'Final'
) AS all_scores
JOIN teams t ON all_scores.team_id = t.team_id
GROUP BY t.team_id, t.nickname
ORDER BY avg_pts DESC
LIMIT 100
```

SQL 查询结果节选：

SQL 查询结果

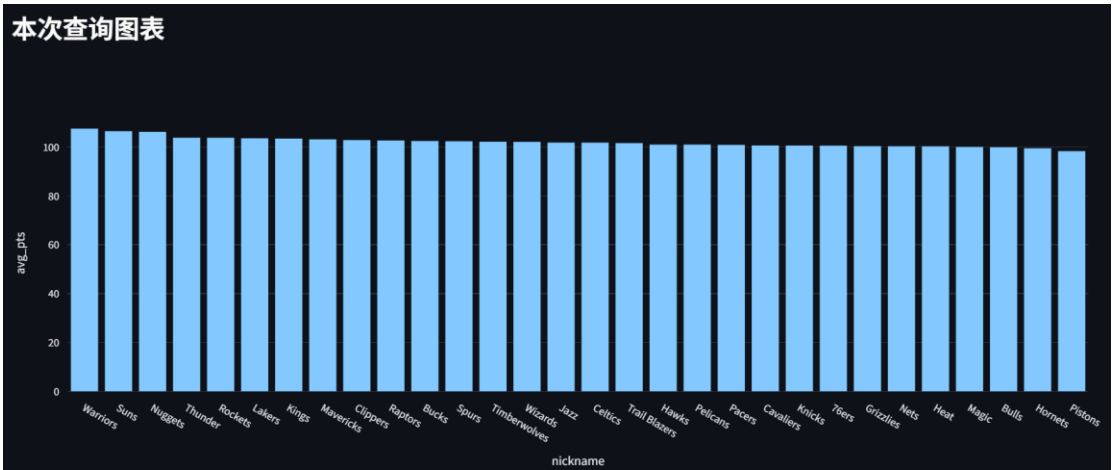
	nickname	team_id	avg_pts
0	Warriors	1610612744	107.5011
1	Suns	1610612756	106.4683
2	Nuggets	1610612743	106.1947
3	Thunder	1610612760	103.7938
4	Rockets	1610612745	103.7841
5	Lakers	1610612747	103.5612
6	Kings	1610612758	103.4372
7	Mavericks	1610612742	103.136
8	Clippers	1610612746	102.8353
9	Raptors	1610612761	102.6687
10	Pistons	1610612762	102.6687

自然语言解释：

智能体分析结论

根据查询结果，场均得分最高的球队是金州勇士队，平均每场得到约107.5分，其次是菲尼克斯太阳队（约106.5分）和丹佛掘金队（约106.2分）。排名前五的球队还包括俄克拉荷马城雷霆队（约103.8分）和休斯顿火箭队（约103.8分），整体来看，只有这三支球队场均得分超过106分，其余球队均低于104分。

本次查询图表：



案例 4：球员单场得分 Top 10

自然语言问题：单场得分最高的 10 名球员是谁？

输入中文业务问题

单场得分最高的 10 名球员是谁？

运行分析

智能体生成或等价执行 SQL：

生成的 SQL

```
SELECT player_name, pts FROM player_game_stats ORDER BY pts DESC LIMIT 10
```

SQL 查询结果节选：

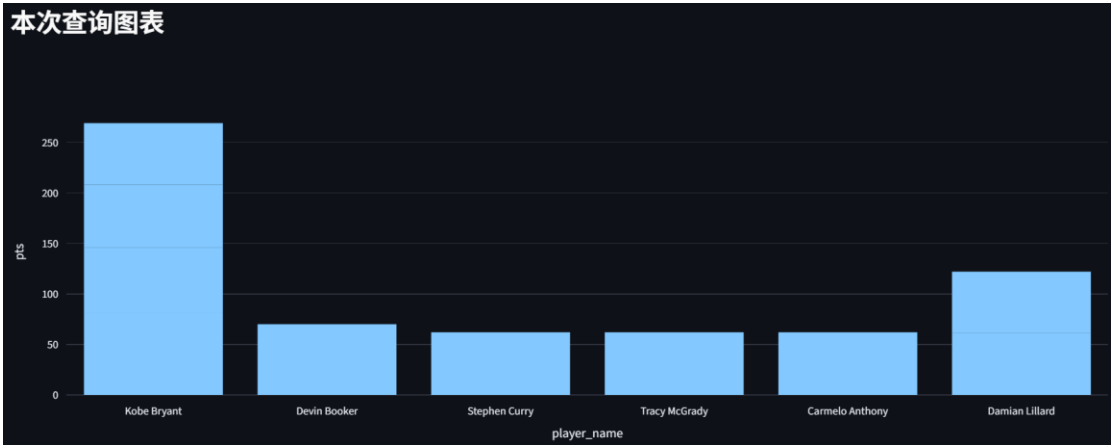
SQL 查询结果		
	player_name	pts
0	Kobe Bryant	81
1	Devin Booker	70
2	Kobe Bryant	65
3	Stephen Curry	62
4	Kobe Bryant	62
5	Tracy McGrady	62
6	Carmelo Anthony	62
7	Damian Lillard	61
8	Damian Lillard	61
9	Kobe Bryant	61

自然语言解释：

智能体分析结论

根据查询结果，单场得分最高的10个记录依次为：科比·布莱恩特（81分）、德文·布克（70分）、科比·布莱恩特（65分）、斯蒂芬·库里（62分）、科比·布莱恩特（62分）、特雷西·麦克格雷迪（62分）、卡梅罗·安东尼（62分）、达米安·利拉德（61分）、达米安·利拉德（61分）、科比·布莱恩特（61分）。其中，科比·布莱恩特一人占据4次，显示出其顶尖的得分爆发力。另外，达米安·利拉德也贡献了两次61分，位列榜单末尾。

本次查询图表：



案例 5：赛季场均总得分趋势

自然语言问题：不同赛季的场均总得分趋势如何变化？

输入中文业务问题

不同赛季的场均总得分趋势如何变化？

运行分析

智能体生成或等价执行 SQL：

生成的 SQL

```
SELECT season, AVG(total_points) AS avg_total_points FROM games GROUP BY season ORDER BY season
LIMIT 100
```

SQL 查询结果节选：

SQL 查询结果

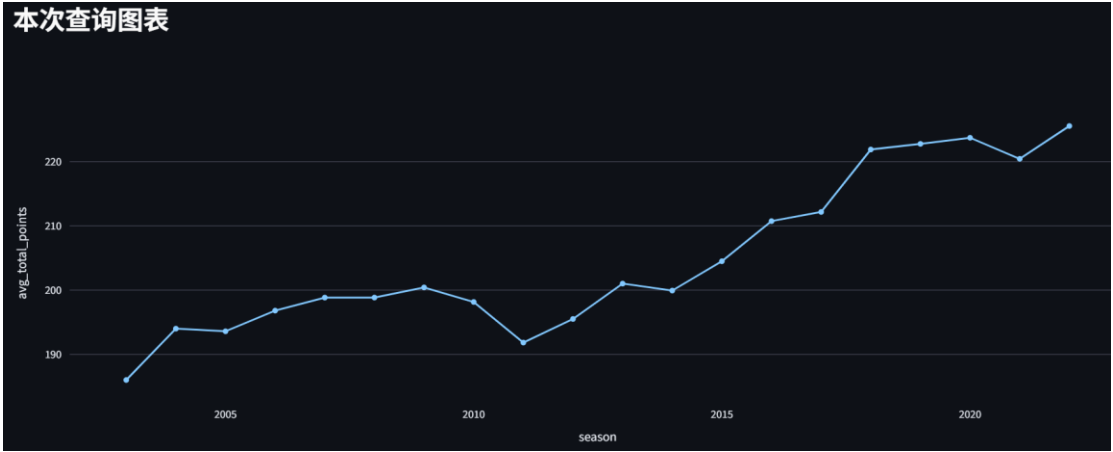
	season	avg_total_points
0	2003	186
1	2004	193.9868
2	2005	193.5789
3	2006	196.7935
4	2007	198.8228
5	2008	198.8267
6	2009	200.4003
7	2010	198.1414
8	2011	191.8216
9	2012	195.4958

自然语言解释：

智能体分析结论

根据查询结果，从2003到2022赛季，场均总分整体呈现显著上升趋势，从186分增长至约225.5分。尤其在2015赛季后，得分提升速度加快，表明联盟进攻效率和节奏持续提升。尽管个别赛季（如2011年）出现短暂回落，但长期方向为增长，最新赛季（2022年）已创下历史最高水平。

本次查询图表：



九、大模型使用说明

本项目使用 DeepSeek API 作为大模型服务，主要用于自然语言问题理解、SQL 生成和查询结果解释。API Key 不写入代码、不写入报告、不提交到 Git，只通过本地 .env 文件配置。

项目	说明
模型名称	默认 deepseek-v4-flash，可通过 DEEPSEEK_MODEL 环境变量切换。
接口方式	OpenAI 兼容方式，base_url 为 https://api.deepseek.com。
使用环节	自然语言问题理解、SQL 生成、SQL 查询结果中文解释。

关键控制	temperature=0，提示词要求只输出 SQL，SQL 安全模块二次检查。
人工修改	人工编写数据清洗、建库、SQL 安全、错误处理、界面展示和报告内容。

十、运行步骤与复现说明

环境配置：

```
python3 -m venv .venv

source .venv/bin/activate

python -m pip install -r requirements.txt
```

如果需要启用 DeepSeek 智能问答，需要复制 .env.example 为 .env，并填写新的 DeepSeek API Key。

```
cp .env.example .env

# 编辑 .env

DEEPSEEK_API_KEY=你的新 key

DEEPSEEK_BASE_URL=https://api.deepseek.com

DEEPSEEK_MODEL=deepseek-v4-flash
```

数据配置及项目运行：

```
cd 项目所在目录

source .venv/bin/activate

python src/download_data.py

python src/preprocess.py

python src/build_database.py

python src/smoke_test.py

streamlit run src/app.py
```

十一、测试与验收结果

测试项	命令/方式	结果
依赖导入	python src/smoke_test.py	通过，pandas、SQLAlchemy、Streamlit、LangChain、OpenAI、Plotly 均可导入。
数据库检查	python src/smoke_test.py	通过，检测到 6 张表和 26622 场比赛。
SQL 安全检查	python src/smoke_test.py	通过，DROP TABLE 等危险语句被拒绝。
Streamlit 页面	streamlit run src/app.py	通过，本地健康检查返回 ok，固定图表可展示。
无 API Key 情况	python src/agent.py	通过，提示配置 DEEPSEEK_API_KEY，不直接崩溃。

十二、总结

本实验完成了从 NBA 公开数据集下载、pandas 清洗、SQLite 建模、SQL 查询验证，到 LangChain + DeepSeek Data Agent 和 Streamlit 可视化展示的完整流程。系统能够根据中文业务问题生成只读 SQL，执行真实数据库查询，并基于查询结果生成中文解释。实验过程中重点保证了数据链路可复现、SQL 查询结果真实、API Key 不泄露、危险 SQL 被拦截，符合数据库系统原理课程对数据处理、建库、查询、智能体和可视化展示的综合要求。

附录 A：项目文件说明

文件	说明
src/download_data.py	下载 Kaggle NBA 数据集，支持 Kaggle CLI 和公开 API fallback。
src/preprocess.py	pandas 清洗脚本，输出 processed CSV 和 data_profile.md。
src/build_database.py	SQLite 建库、导入、索引和验证查询脚本。
src/sql_safety.py	只读 SQL 安全检查，拦截危险关键字和多语句。
src/agent.py	LangChain + DeepSeek SQL Agent 封装。
src/app.py	Streamlit 页面，展示问题、SQL、结果、解释和图表。
src/smoke_test.py	本地烟雾测试，不调用大模型。

附录 B：运行截图

以下截图用于证明项目可以在本地环境实际运行。图表截图已在正文“Streamlit 界面与可视化”部分展示，本附录补充烟雾测试和 DeepSeek Agent 端到端查询链路截图。

```
● (datawork) PS D:\class\大三下\数据库\实验\实验7\BBB-master> python src/smoke_test.py
Python package imports: ok
SQLite database: ok (6 tables, 26622 games)
SQL safety: ok
Smoke test passed.
```

图 B-1 本地烟雾测试运行截图。截图显示依赖导入、数据库表数量、比赛数量和 SQL 安全检查均通过。

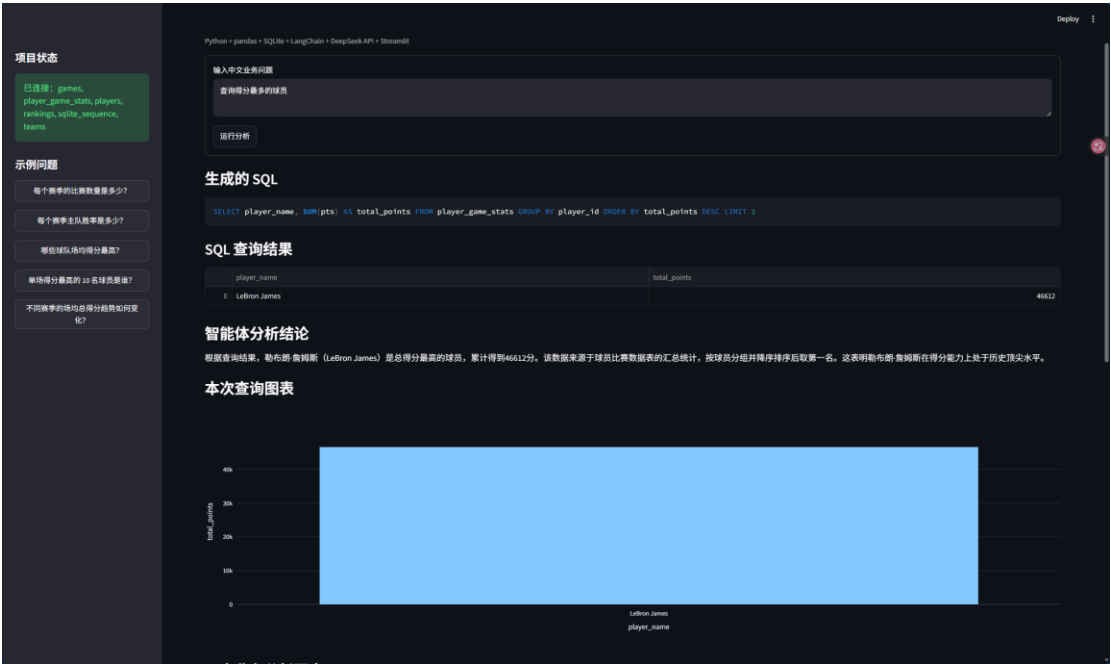


图 B-2 DeepSeek Agent 自然语言查询运行截图。